

Sistema de **GE**stión de **Fl**ota de **V**ehículos **El**éctricos (**SIGEFVE**)

Maestría en Sistemas Computacionales

Materia:

Tecnologías de Programación

Catedrático:

Dr. Ulises Juárez Martínez

Alumnos:

Aldo Iván López Vivanco

Jessica Hernández Mayahua

Coré Husáí Martínez Belmonte

Tabla de contenido

Tabla de contenido	2
Introducción	3
Objetivo general	4
Contexto del negocio	4
Distribución de responsabilidades por lenguaje	4
Requisitos funcionales	4
Módulo Java: gestión de flota	4
Módulo Python: análisis y alertas	5
Módulo Go: API Gateway	5
Frontend (HTML/JavaScript o React)	6
Requisitos no funcionales	6
Patrón arquitectónico	7
Diagramas	9
Diagrama de casos de uso	9
Diagrama de clases - Microservicio Java	10
Diagrama de clases - Microservicio Python	11
Diagrama de clases - Microservicio Go	11
Base de datos por cada microservicio	12
Modelo relacional de base de datos - Java	12
Modelo relacional de base de datos - Python	13
Modelo relacional de base de datos - Go	14
Vistas	16
Referencias	22

Introducción

El presente documento detalla la arquitectura de software del Sistema de Gestión de Flota de Vehículos Eléctricos (SIGEFVE). El propósito de este entregable es documentar el diseño y la estructuración lógica de una solución capaz de administrar, monitorear y optimizar una flota de vehículos eléctricos mediante la integración de múltiples lenguajes de programación (Java, Python y Go).

Para responder a las necesidades de un negocio de reparto urbano con requerimientos de telemetría en tiempo real y alta disponibilidad, se ha optado por un patrón arquitectónico de microservicios. Esta decisión, fundamentada en autores como Guadaño (2024) y Newman (2015), permite descomponer el sistema en unidades autónomas y desplegables independientemente, resolviendo problemas complejos de coordinación y escalabilidad.

La arquitectura propuesta distribuye las responsabilidades del sistema en tres pilares tecnológicos interconectados mediante API REST:

API Gateway (Go): Implementa el patrón de punto de entrada único, centralizando la autenticación vía JWT, el enrutamiento de peticiones y el control de tráfico (rate limiting).

Gestión de Flota (Java): Modela el núcleo del negocio y la persistencia de datos transaccionales de vehículos y rutas bajo un enfoque orientado a objetos.

Análisis y Alertas (Python): Funciona como un sistema reactivo que implementa el patrón de diseño Observer para el procesamiento asíncrono de telemetría y la generación de notificaciones críticas.

A lo largo del documento se presentan los artefactos de diseño esenciales, incluyendo diagramas de casos de uso, diagramas de clases para cada microservicio y modelos relacionales de base de datos independientes. Estos elementos demuestran la coherencia entre el modelo conceptual y los requisitos funcionales, asegurando un diseño robusto, modular y alineado con los estándares de la ingeniería de software moderna.

Objetivo general

Implementar un sistema aplicando correctamente el modelo de objetos integrando Java, Python y Go mediante arquitectura de microservicios.

Contexto del negocio

Una empresa de reparto urbano tiene una flota de 15 vehículos eléctricos (vans, bicicletas eléctricas y motos).

Cada vehículo tiene:

- Datos de telemetría: Nivel de batería, ubicación GPS, temperatura del motor, velocidad.
- Estado operativo: Disponible, En Ruta, En Mantenimiento, Cargando.
- Información de ruta: Ruta asignada, entregas pendientes, entregas completadas.

El sistema debe:

1. Registrar telemetría de cada vehículo periódicamente (cada 15 segundos).
2. Asignar rutas de entrega a vehículos disponibles.
3. Generar alertas cuando la batería esté baja o haya problemas mecánicos.
4. Calcular estadísticas de rendimiento de la flota.
5. Mostrar estado actual y ubicación de vehículos en un dashboard web.

Distribución de responsabilidades por lenguaje

Lenguaje	Responsabilidad	Tecnologías
Java	Gestión de vehículos, telemetría, rutas de entrega	Java con servidor HTTP básico, PostgreSQL, JDBC
Python	Análisis de rendimiento, sistema de alertas, reportes	Flask, SQLite, Threading
Go	API Gateway, autenticación, enrutamiento	Gin, JWT
JavaScript	Interfaz gráfica de usuario	React.js

Requisitos funcionales

Módulo Java: gestión de flota

Funcionalidades mínimas:

1. CRUD de vehículos (crear, consultar, actualizar, eliminar).
2. Registro automático de telemetría cada 15 segundos (simulado).
3. Consulta de historial de telemetría por vehículo.
4. Asignación de rutas a vehículos disponibles.
5. Cambio de estado operativo de vehículos.
6. Consulta de entregas completadas por vehículo.
- 7.

Entidades principales:

- Vehiculo (clase abstracta).
- VehiculoElectrico (hereda de Vehiculo).
- Van, BicicletaElectrica, MotoElectrica (heredan de VehiculoElectrico).
- Telemetria (almacena datos históricos de sensores).
- Ruta (representa una ruta de entregas).
- Entrega (representa una entrega individual).
- EstadoVehiculo (enum: DISPONIBLE, EN_RUTA, MANTENIMIENTO, CARGANDO).

Módulo Python: análisis y alertas

Funcionalidades mínimas:

1. Recibir eventos de telemetría desde Java.
2. Calcular estadísticas de rendimiento por vehículo:
 - a. Kilómetros recorridos promedio.
 - b. Eficiencia de batería.
 - c. Entregas completadas.
3. Generar alertas automáticas:
 - a. Batería baja (< 20%).
 - b. Temperatura alta del motor (> 70°C).
4. Listar alertas activas por prioridad.
5. Exportar reporte de rendimiento en CSV.

Entidades principales:

- Alerta (clase base)
- AlertaUrgente (herencia múltiple: Alerta + Notificable + Registrable).
- AlertaMantenimiento (herencia múltiple).
- ProcesadorEstadisticas (clase abstracta).
- AnalizadorRendimiento (procesa métricas de vehículos).
- ObservadorTelemetria (patrón Observer).
- GestorEventos (maneja eventos asincrónicamente con threading).

Módulo Go: API Gateway

Funcionalidades mínimas:

1. Login con usuario/contraseña → genera token JWT.

2. Validar token en cada petición a servicios protegidos.
3. Enrutar peticiones a Java o Python según el path.
4. Registrar log de todas las peticiones.
5. Limitar peticiones por IP (rate limiting básico).
6. Endpoint de salud (health check) para monitoreo.

Frontend (HTML/JavaScript o React)

Funcionalidades mínimas:

1. Pantalla de login.
2. Dashboard con:
 - a. Mapa simple mostrando ubicación de vehículos (puede ser simulado).
 - b. Lista de vehículos con estado, batería y última ubicación.

Requisitos no funcionales

1. Arquitectura:
 - a. Microservicios independientes.
 - b. Comunicación mediante REST.
 - c. Cada servicio con su propia base de datos.
2. Despliegue:
 - a. Todo el sistema debe iniciar con: docker-compose up.
 - b. Debe funcionar en Windows, Linux y macOS.
3. Calidad de código:
 - a. Nombres de variables y métodos en español.
 - b. Comentarios en código complejo.
 - c. Sin código duplicado.
 - d. Manejo de errores apropiado.
4. Seguridad:
 - a. Autenticación con JWT.
 - b. Validación de datos de entrada.
 - c. Contraseñas NO en texto plano en código.

Patrón arquitectónico

De acuerdo con Guadaño (2024), las arquitecturas de microservicios se caracterizan por dar solución a problemas de comunicación y coordinación entre microservicios, autónomos y desplegados de forma independiente, cada uno enfocado en una responsabilidad específica del negocio.

El sistema SIGEFVE cumple con esta definición al separar sus responsabilidades en tres servicios independientes:

- Microservicio Java: responsable de la gestión de la flota y persistencia de datos.
- Microservicio Python: dedicado al análisis de rendimiento y generación de alertas automáticas.
- Microservicio Go (API Gateway): encargado de la autenticación, enrutamiento y registro de peticiones.

Para este problema, se decidió utilizar el patrón de API Gateway, pues actúa como un punto de entrada único para gestionar las solicitudes de clientes en un sistema de microservicios, proporcionando funcionalidades como enrutamiento, autenticación, autorización y agregación de datos.

A nivel interno, el frontend sigue la arquitectura Modelo-Vista-Controlador (MVC), lo que permite una clara separación de responsabilidades:

- Modelo: datos y lógica del dominio (vehículos, alertas, rutas).
- Vista: representación visual de la información al usuario.
- Controlador: gestión de la comunicación entre la vista y el modelo.

Este patrón, según Eric Dashofy, Nenad Medvidović y Richard N. Taylor (2008), promueve la separación, sin interferir con la estructura distribuida de los microservicios.

A continuación se muestra el diagrama se muestra el patrón arquitectónico de microservicios global junto con MVC:

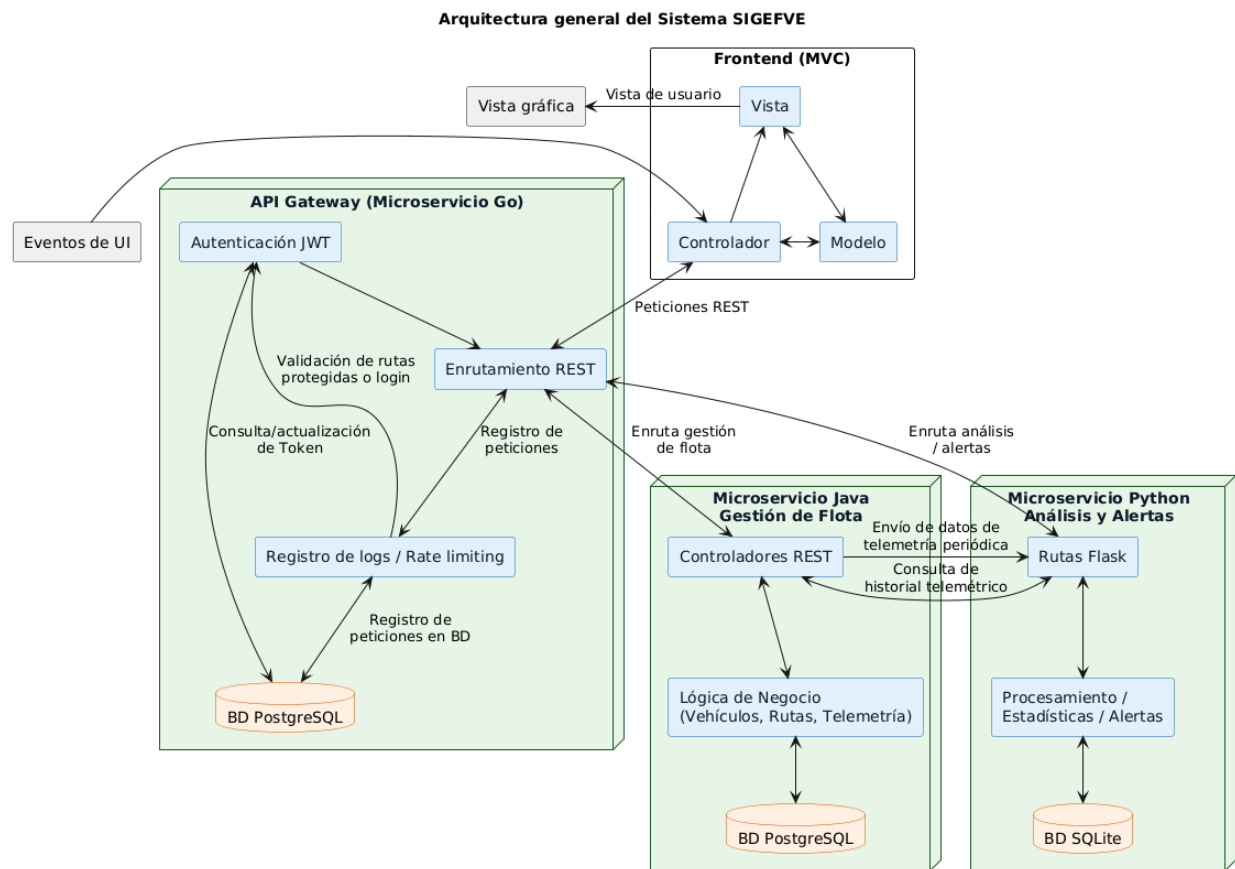


Figura 1. Arquitectura general del sistema.

[Enlace a diagrama](#)

Diagramas

En este apartado se presentan los diagramas elaborados para SIGEFVE, acompañados de un enlace para visualizarlos en la herramienta CASE utilizada para modelarlos y su justificación de diseño, con el propósito de demostrar la coherencia entre el modelo conceptual, la arquitectura propuesta y los objetivos establecidos en las especificaciones del proyecto.

Diagrama de casos de uso

De acuerdo con Booch, Rumbaugh y Jacobson (1999), los diagramas de casos de uso son el punto de partida para modelar los requisitos de un sistema, pues describen su comportamiento desde la perspectiva del usuario. Estos diagramas permiten abstraer los detalles técnicos para centrarse en las funcionalidades esenciales, asegurando que el sistema responda a las necesidades del negocio.

En el diagrama se identifican dos actores principales:

- Administrador, encargado de gestionar la flota, asignar rutas, consultar alertas y generar reportes.
- Conductor, quien interactúa con el sistema para visualizar el estado operativo de los vehículos.

Esta distinción se basa en la recomendación de Pressman (2010), quien sugiere modelar los actores de acuerdo con los roles y responsabilidades reales dentro del contexto del sistema, evitando redundancias y garantizando claridad en las interacciones.

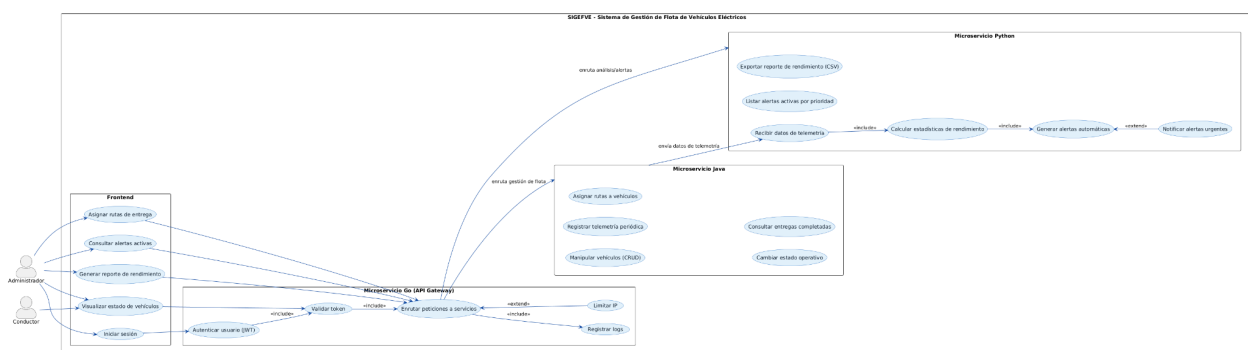


Figura 2. Diagrama de casos de uso.

[Enlace a diagrama](#)

Diagrama de clases - Microservicio Java

El diseño se basa en los principios de orientación a objetos, según los cuales la estructura del software debe modelar fielmente los objetos del dominio real (Booch, Rumbaugh & Jacobson, 1999). En este caso, el dominio corresponde a la gestión de una flota de vehículos eléctricos, y las clases representan los elementos del negocio (vehículos, rutas, telemetría y entregas).

El uso del enum EstadoVehiculo (DISPONIBLE, EN_RUTA, MANTENIMIENTO, CARGANDO) responde al requisito funcional de controlar el estado operativo de los vehículos. La decisión de modelarlo como enumerado se fundamenta en su naturaleza discreta y finita de valores posibles, lo cual incrementa la legibilidad y reduce los errores de codificación (Sommerville, 2016).

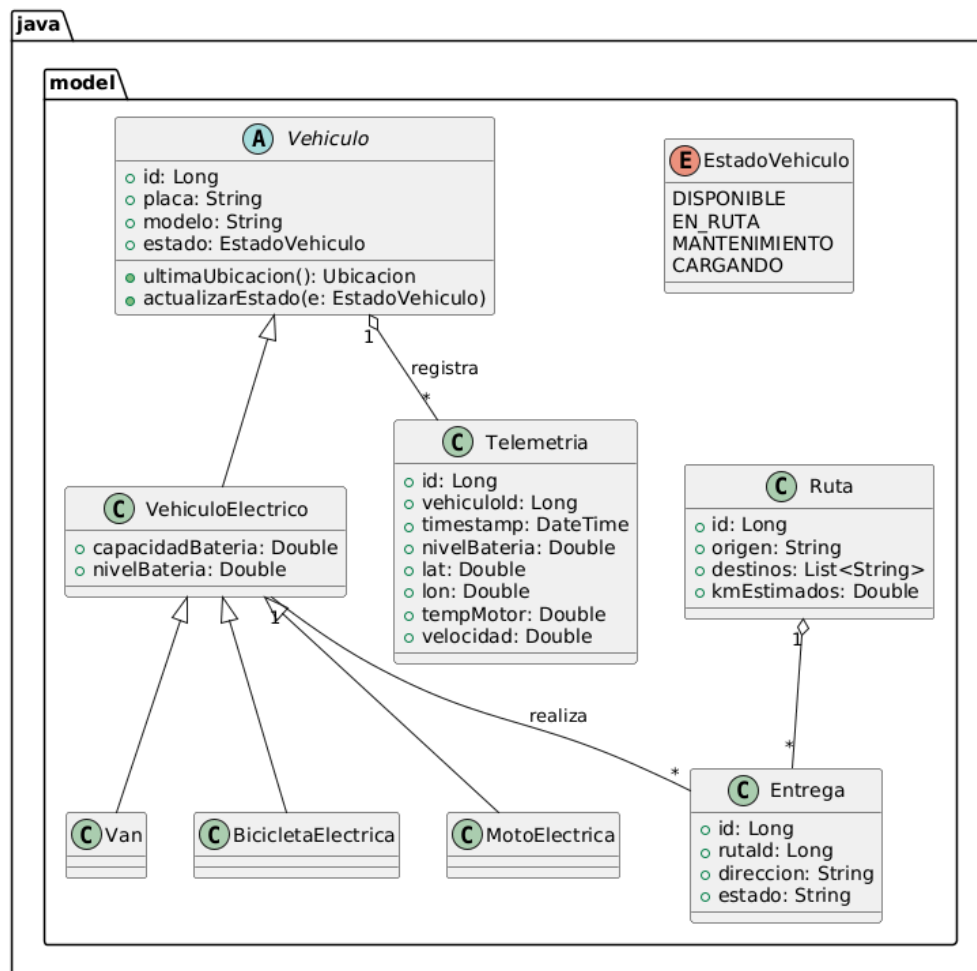


Figura 3. Diagrama de clases del microservicio de Java.

[Enlace a diagrama](#)

Diagrama de clases - Microservicio Python

En particular, el microservicio Python modela el procesamiento de eventos y la generación de alertas como un sistema reactivo y orientado a eventos, implementando el patrón Observer, descrito por Gamma, Helm, Johnson y Vlissides (1995), para manejar la actualización automática entre productores (fuente de telemetría) y consumidores (procesador de datos).

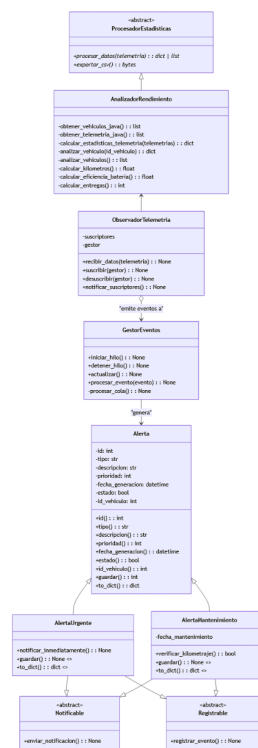


Figura 4. Diagrama de clases del microservicio de Python.

[Enlace a diagrama](#)

Diagrama de clases - Microservicio Go

Aunque Go no es un lenguaje orientado a objetos en el sentido tradicional (no cumple con los 4 pilares mayores de POO), su modelo basado en structs, métodos y composición permite aplicar los principios de diseño modular y encapsulamiento característicos de la orientación a objetos (Donovan & Kernighan, 2015).

Cada struct del diagrama cumple el papel de una clase, mientras que los métodos asociados a dichas estructuras encapsulan el comportamiento del componente.

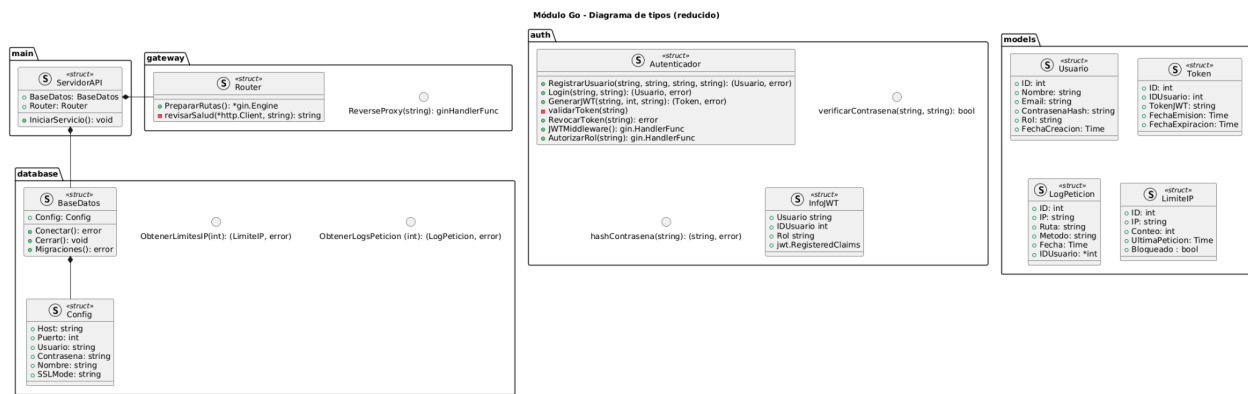


Figura 5. Diagrama de tipos del microservicio de Go.

[Enlace a diagrama](#)

Base de datos por cada microservicio

De acuerdo con Newman (2015), en una arquitectura de microservicios cada servicio debe poseer su propio modelo de datos y su propia persistencia, de modo que sea completamente independiente del resto. Además, esto permite que cada microservicio (Java, Python y Go) pueda actualizar su esquema de base de datos sin afectar a los demás: si varios servicios acceden a la misma base de datos, un cambio en el esquema para un servicio puede romper a otros servicios inesperadamente, lo que va en contra del objetivo de poder modificar y desplegar un servicio de forma independiente.

Modelo relacional de base de datos - Java

La tabla "vehículo" contendrá la información básica y el estado operativo de cada vehículo, aplicando un modelo jerárquico simplificado mediante el atributo tipo, lo cual evita la necesidad de herencia en la base de datos.

"telemetría" almacena lecturas periódicas enviadas por cada vehículo, asegurando trazabilidad histórica mediante el campo timestamp.

“rutas” modela los recorridos, incluyendo la cantidad de entregas que va a realizar, la fecha de inicio y la fecha de fin..

“entrega” establece la relación entre un vehículo y su ruta activa, representando la unidad básica del proceso logístico.

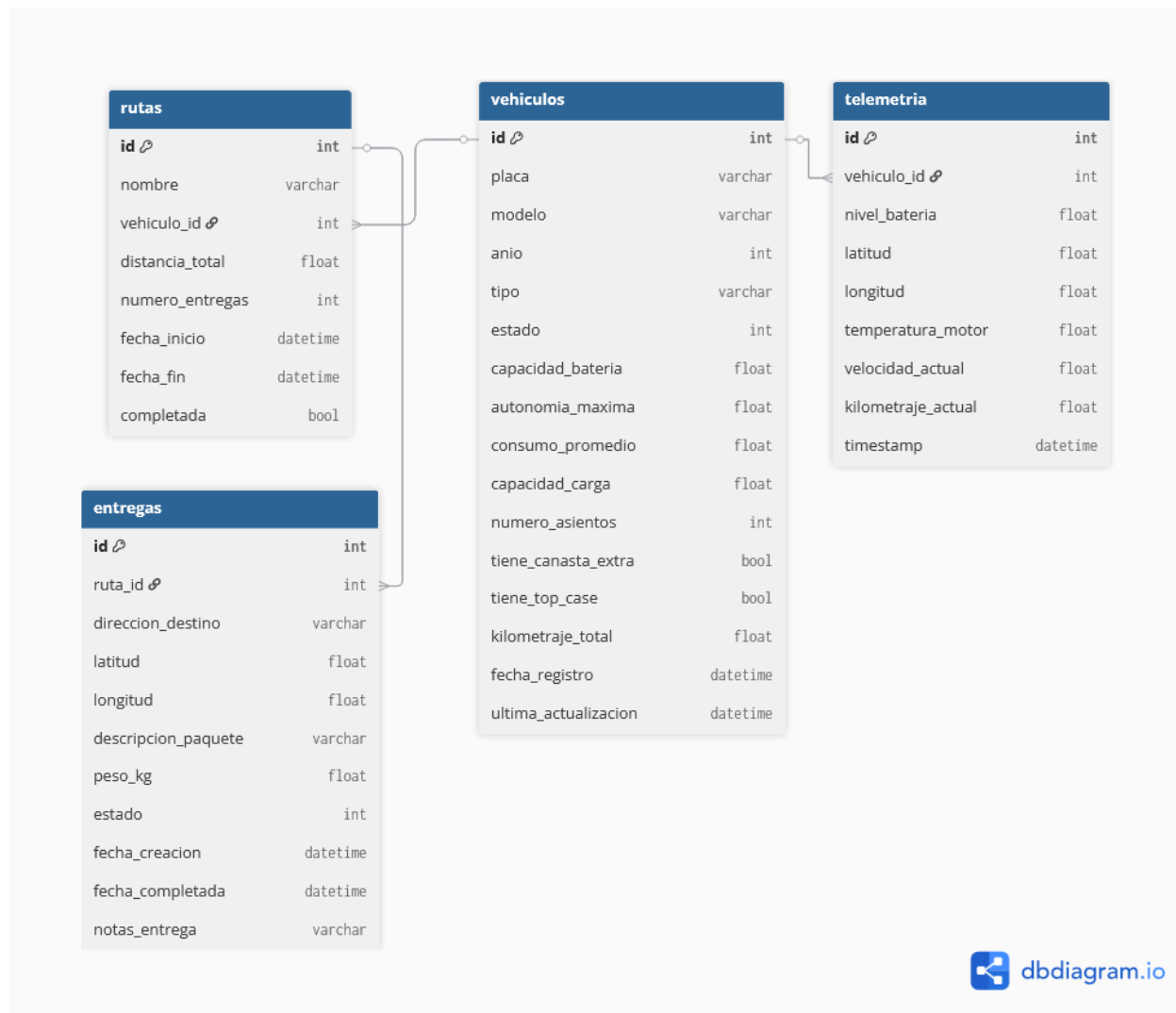


Figura 6. Modelo relacional del microservicio Java.

[Enlace a diagrama](#)

Modelo relacional de base de datos - Python

Estadística almacena los resultados del análisis de datos de telemetría (kilometraje, eficiencia y entregas completadas), permitiendo obtener métricas históricas por vehículo.

Alerta constituye la entidad base para todos los tipos de alertas, con atributos que permiten clasificarlas por prioridad y estado.

Las tablas especializadas AlertaMantenimiento y AlertaUrgente extienden la funcionalidad de Alerta mediante claves foráneas únicas (id_alerta), implementando el principio de herencia en el modelo relacional.

- AlertaMantenimiento añade información sobre la fecha de mantenimiento programado.
- AlertaUrgente incorpora un mensaje personalizado para notificación inmediata.

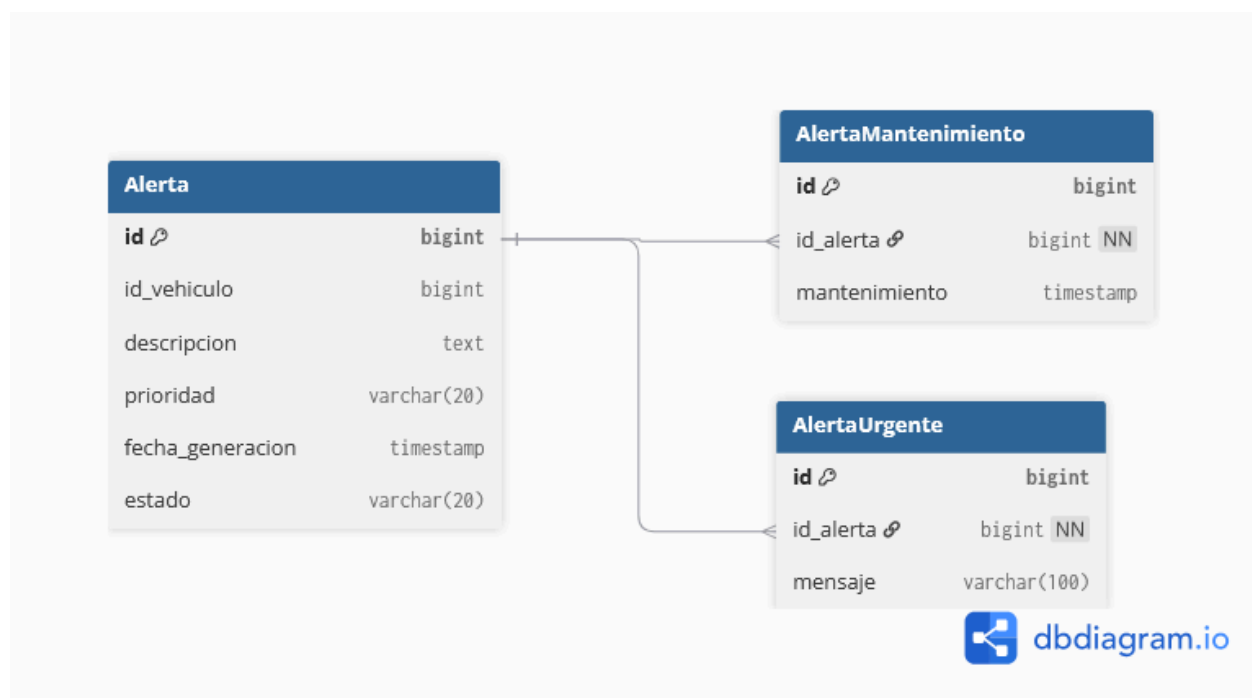


Figura 7. Modelo relacional del microservicio Python.

[Enlace a diagrama](#)

Modelo relacional de base de datos - Go

Usuario almacena las credenciales y roles de los actores del sistema (Administrador y Conductor), garantizando integridad y unicidad mediante índices en nombre y email.

Token gestiona los JWT generados al iniciar sesión, registrando su vigencia y relación con el usuario correspondiente, lo que permite implementar control de sesiones y auditoría de seguridad.

LogPetición registra las solicitudes al gateway, incluyendo dirección IP, ruta y método, lo que respalda la trazabilidad del uso del sistema.

LimitIP implementa un mecanismo básico de rate limiting, controlando la frecuencia de acceso por IP y marcando aquellas temporalmente bloqueadas.

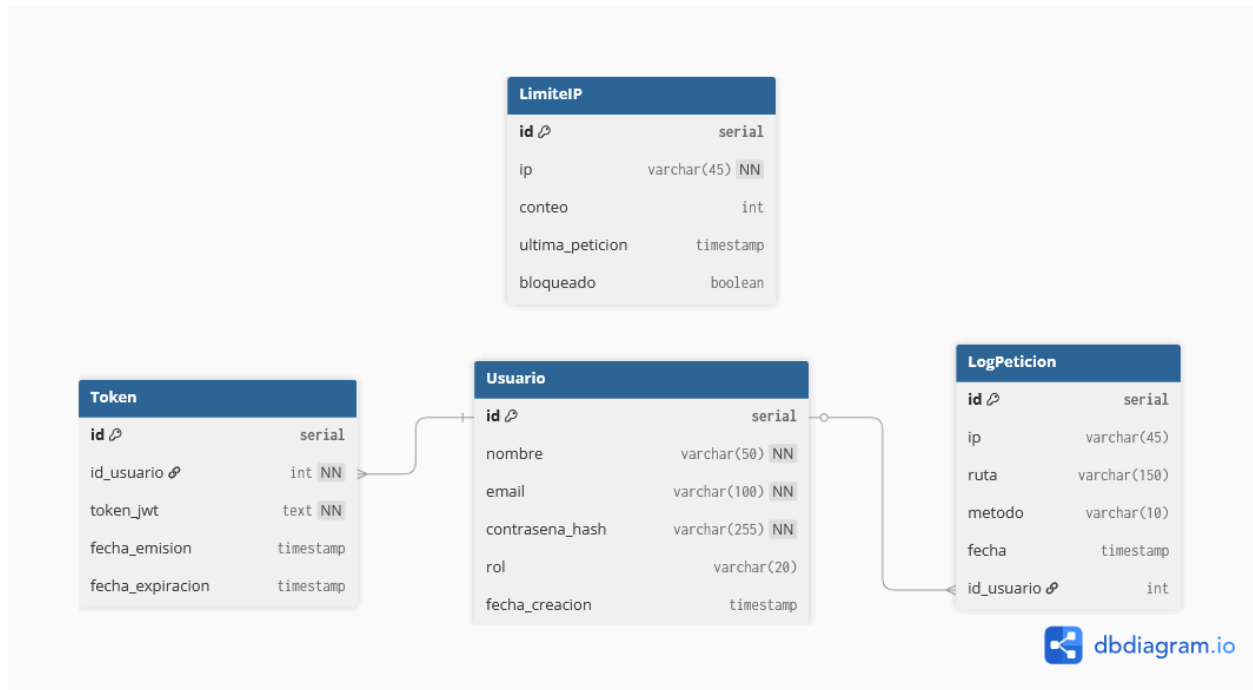


Figura 8. Modelo relacional del microservicio Go.

[Enlace a diagrama](#)

Vistas

En esta sección se detallan las vistas del sistema.

Login

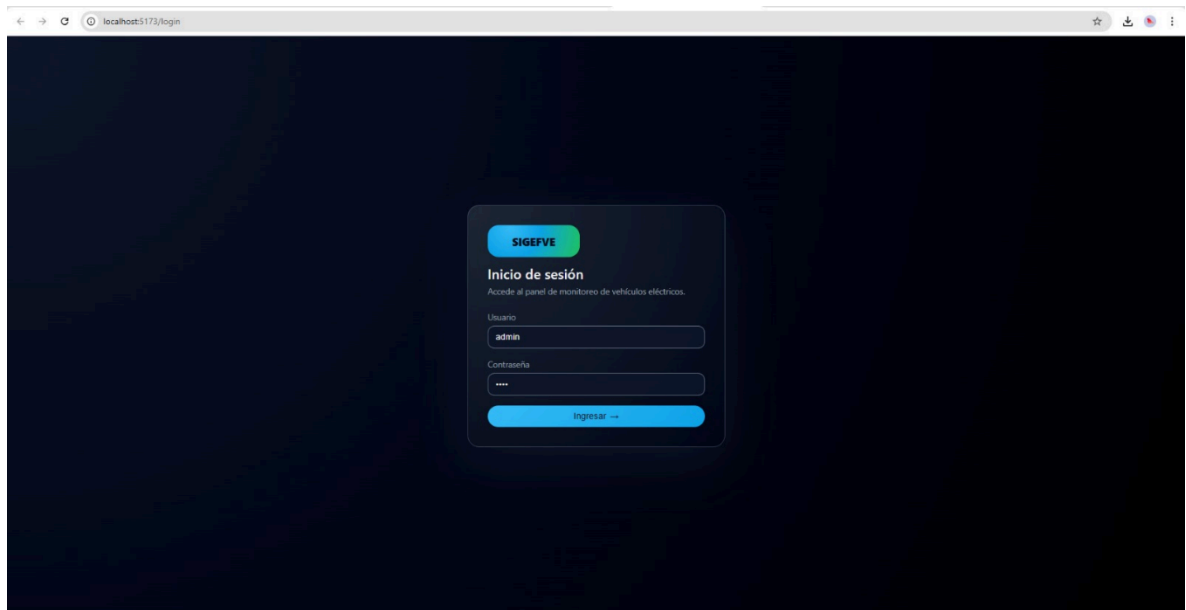


Figura 9. Inicio de sesión.

Dashboard

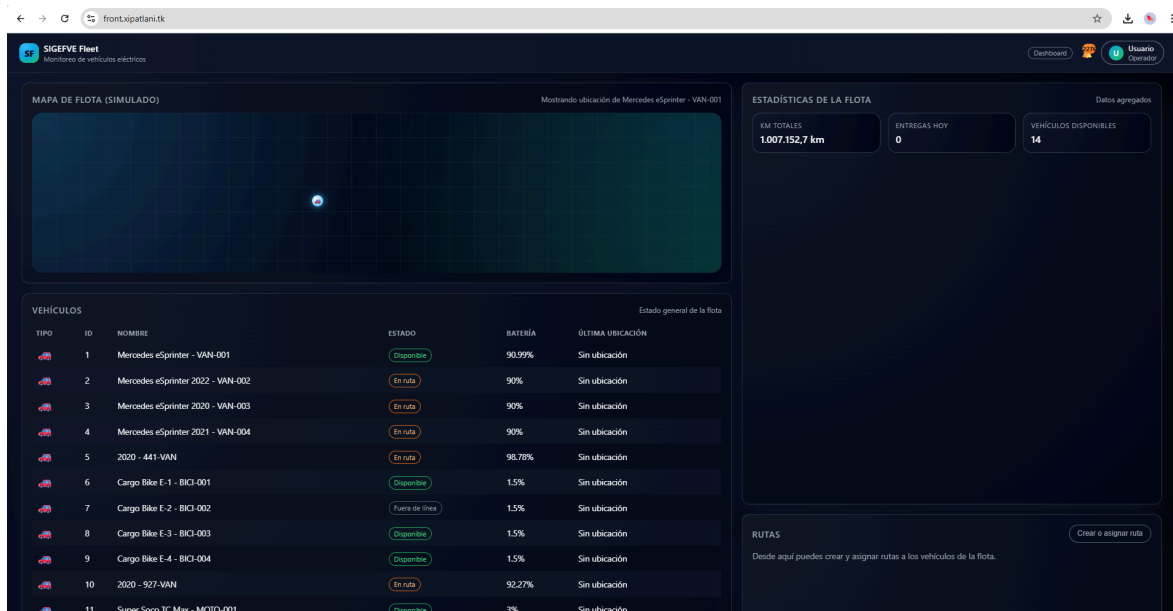


Figura 10. Página principal.

Lista de vehículos

SIGEFVE Fleet
Monitoreo de vehículos eléctricos

VEHÍCULOS Estado general de la flota

TIPO	ID	NOMBRE	ESTADO	BATERÍA	ÚLTIMA UBICACIÓN
	1	Mercedes eSprinter - VAN-001	Disponible	90.99%	Sin ubicación
	2	Mercedes eSprinter 2022 - VAN-002	En ruta	90%	Sin ubicación
	3	Mercedes eSprinter 2020 - VAN-003	En ruta	90%	Sin ubicación
	4	Mercedes eSprinter 2021 - VAN-004	En ruta	90%	Sin ubicación
	5	2020 - 441-VAN	En ruta	98.78%	Sin ubicación
	6	Cargo Bike E-1 - BICI-001	Disponible	1.5%	Sin ubicación
	7	Cargo Bike E-2 - BICI-002	Fuera de línea	1.5%	Sin ubicación
	8	Cargo Bike E-3 - BICI-003	Disponible	1.5%	Sin ubicación
	9	Cargo Bike E-4 - BICI-004	Disponible	1.5%	Sin ubicación
	10	2020 - 927-VAN	En ruta	92.27%	Sin ubicación
	11	Super Soco TC Max - MOTO-001	Disponible	3%	Sin ubicación
	12	2020 - 428-VAN	Disponible	97.01%	Sin ubicación
	13	2020 - 76-VAN	Disponible	94.52%	Sin ubicación
	14	Super Soco TC Max - MOTO-004	En ruta	3%	Sin ubicación
	15	Super Soco TC Max - MOTO-005	Fuera de línea	3%	Sin ubicación
	18	string - 5011-string	Disponible	50%	Sin ubicación
	19	2020 - 63-VAN	Disponible	93.92%	Sin ubicación
	20	2020 - 185-VAN	Disponible	91.96%	Sin ubicación
	21	2020 - 605-VAN	Disponible	88.75%	Sin ubicación
	22	2020 - 478-VAN	Disponible	97.18%	Sin ubicación

Figura 11. Dashboard.

Detalles de vehículos

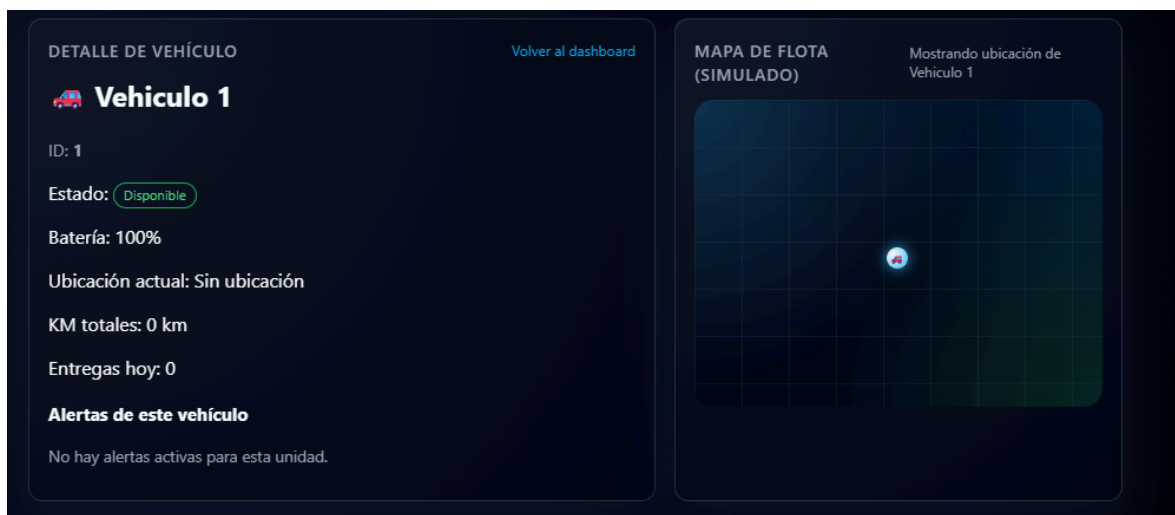


Figura 12. Listado de vehículos.

Asignación de ruta

The form is titled 'Crear / Asignar Ruta'. It contains the following fields and controls:

- Origen:** A text input field with the placeholder 'Dirección origen'.
- Destinos:** A section containing a text input field for 'Destino 1' and a green button labeled 'Agregar destino'.
- Vehículo a asignar:** A dropdown menu with the placeholder '-- Selecciona un vehículo --'.
- Fecha/Hora estimada:** A date and time picker field with the placeholder 'dd/mm/aaaa --:--:--'.
- Crear y asignar:** A blue button at the bottom of the form.

Figura 13. Creación de ruta.

Crear/asignar ruta

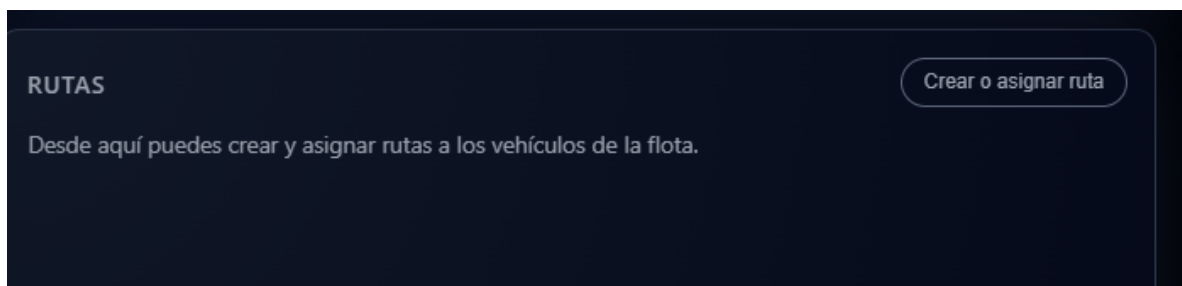


Figura 14. Listado de alertas.

Lista de alertas

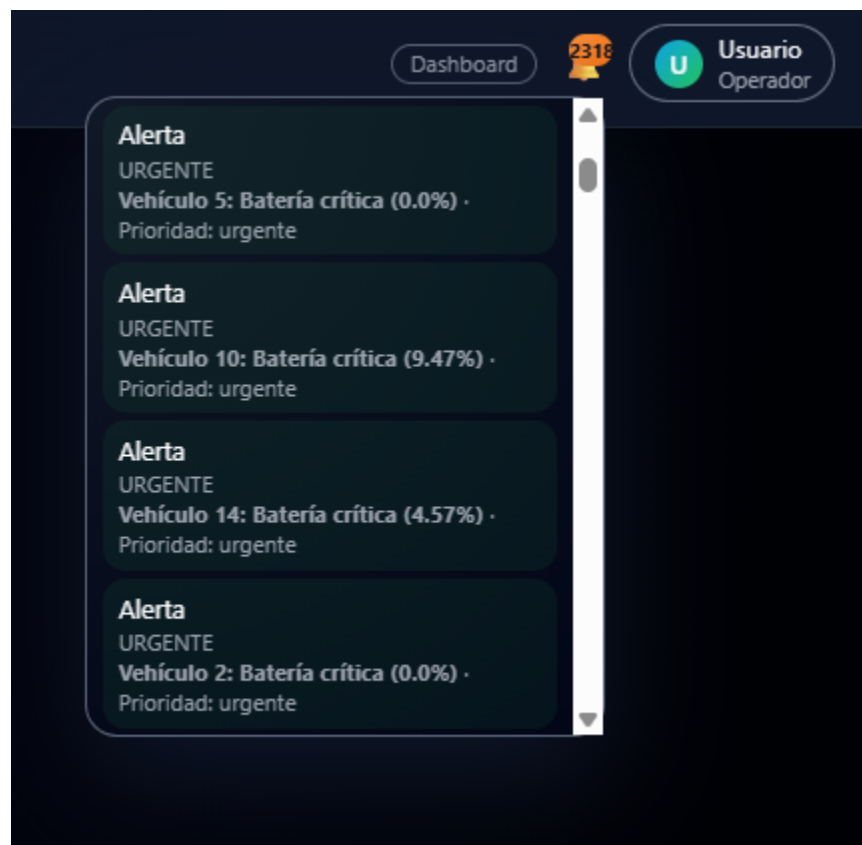


Figura 16. Vista de reporte específico.

Referencias

Booch, G., Rumbaugh, J., & Jacobson, I. (1999). The Unified Modeling Language User Guide. Addison-Wesley.

Pressman, R. S. (2010). Software Engineering: A Practitioner's Approach (7th ed.). McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Guadaño, Y. (2024). Patrones de arquitectura de microservicios: SAGA, API Gateway y Service Discovery. Paradigma. Disponible en:
<https://www.paradigmadigital.com/dev/patrones-arquitectura-microservicios-saga-api-gateway-service-discovery/>

Dashofy, E., Medvidović, N., & Taylor, R. (2008). Software Architecture: Foundations, Theory, and Practice. John Wiley & Sons.

Donovan, A. A., & Kernighan, B. W. (2015). The Go Programming Language. Addison-Wesley.

Newman, S. (2015). Building Microservices: Designing Fine-Grained Systems. O'Reilly Media.

Refactoring Guru. (s. f.) Observer. Disponible en
<https://refactoring.guru/es/design-patterns/observer>